

L05: Support Vector Machines

CSci 574 Machine Learning (Géron) Ch. 5

Derek Harter

Department of Computer Science
East Texas A&M University

Fall 2025

Large Margin Classification

When two classes can be separated by a line (hyperplane).

- They are **linearly separable**

SVM classifier fits the widest possible street.

- This is called **large margin classification**

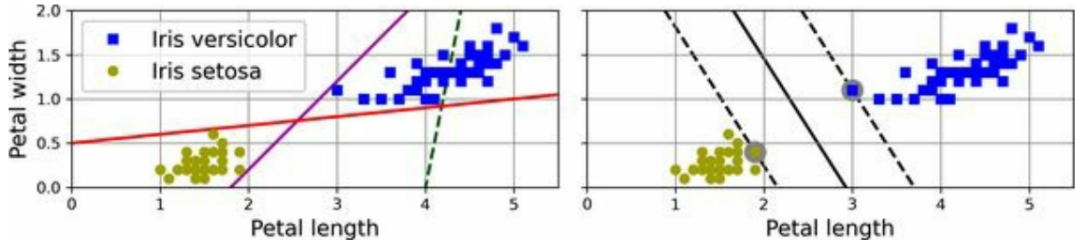


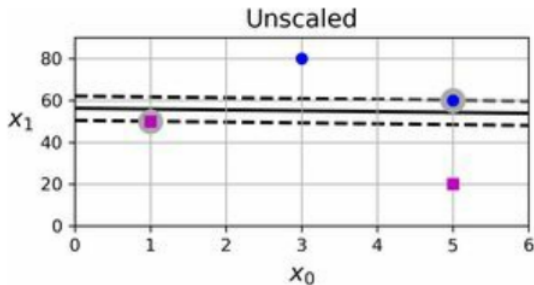
Figure 1: Large margin classification (Géron, 2023, pg.287)

Linear Support Vector Machine

- Notice that adding more training instances “off the street” will not affect the decision boundary at all.
 - It is fully determined (or “supported”) by the instances located on the edge of the street
 - These instances are called the **support vectors**
- A Support Vector Machine (SVM) is a machine learning model capable of performing linear or nonlinear classification and regression.
- It works by identifying support vectors to determine the largest margin decision boundary

SVMs are Sensitive to Scaling

In the left plot vertical scale is much larger than the horizontal scale, gives a almost horizontal large margin with these support vectors.



In the right plot after standard feature scaling, the decision boundary had a much different slope.

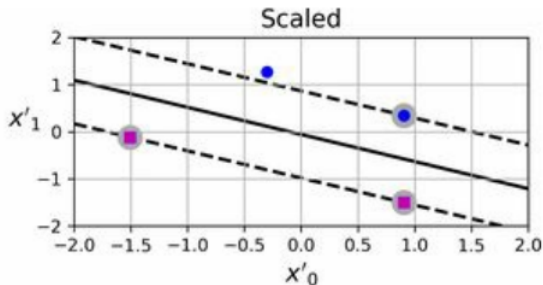


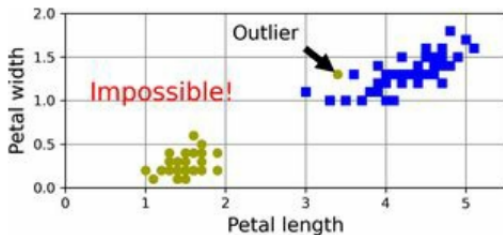
Figure 2: SVM sensitivity to feature scales. (Géron, 2023, pg.288)

Soft Margin Classification

If we strictly require all instances off street and on correct side of margin, this is called **hard margin classification**.

Only possible when data are linearly separably.

- Figure 3 left, impossible to find hard margin, not linearly separable.



Also sensitive to outliers.

- Figure 3 right, decision boundary ends up very different if there is an outlier here, might not generalize.

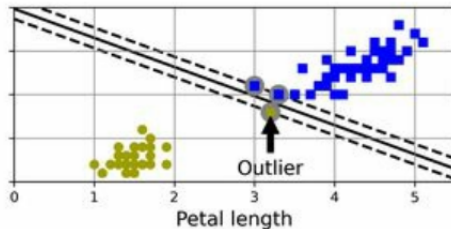


Figure 3: Hard margin sensitivity to outliers. (Géron, 2023, pg.288)

Soft Margin Classification

- To avoid both, find balance between keeping street as large as possible and limiting the **margin violations**
 - This is **soft margin classification**
- SVM use regularization hyperparameter C
 - When C is low, large margin, more margin violations allowed, but if too low can cause underfitting.
 - When C is high, small margins, less margin violations allowed, but can overfit and be sensitive to outliers

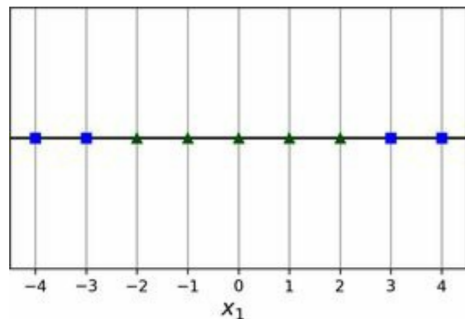
SVM C Regularization

If your SVM model is overfitting, you can regularize it by reducing C , leading to softer margins which will tend to be better at generalizing performance to unseen data.

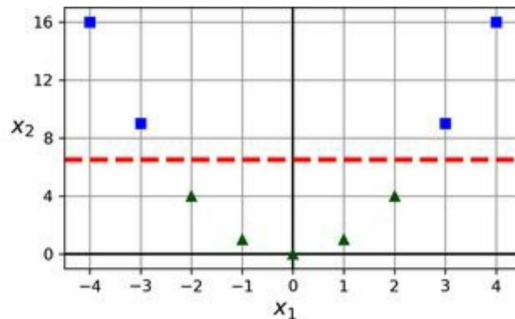


Nonlinear SVM Classification

- Many datasets are not even close to being linearly separable.
- One approach is to add more polynomial feature combinations of the original features.
- This can result in transforming data into a linearly separable dataset.



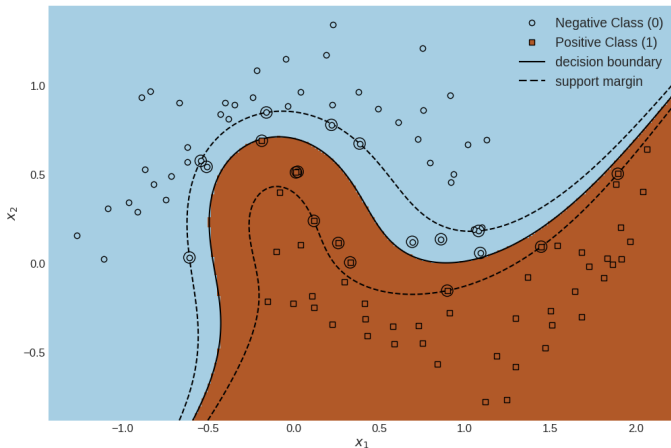
Single feature x_1 is not linearly separable.



But if we add new polynomial feature $x_2 = (x_1)^2$, resulting 2D dataset is linearly separable.

Adding Nonlinear Features by Hand

As we have done previously for logistic regression, add in nonlinear polynomial features by hand.



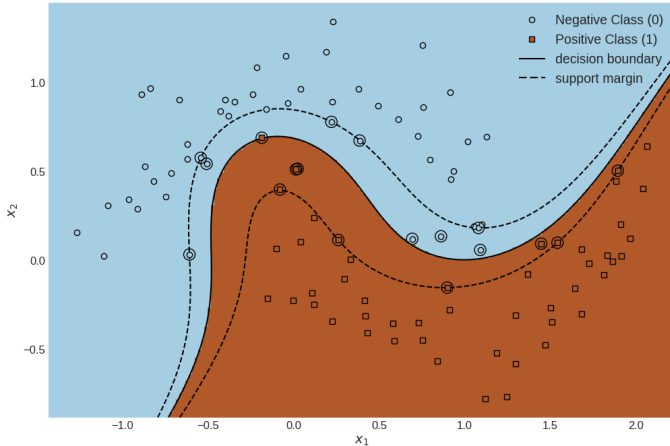
```
# adding degree 3 polynomials combinations  
# to 2 feature nonlinear dataset  
# makes resulting nonlinear features  
# linearly separable  
X, y = make_moons(n_samples=100, noise=0.15)  
  
polynomial_svm_clf = Pipeline([  
    ("poly_features", PolynomialFeatures(degree=3)),  
    ("scaler", StandardScaler()),  
    ("clf", SVC(kernel='linear', C=10, max_iter=10000))  
)  
  
polynomial_svm_clf.fit(X, y)
```

Figure 4: Linear SVM classifier using polynomial features.

Polynomial Kernel, the **Kernel Trick**

- Adding polynomial features can work with all sorts of ML (not just SVMs)
 - But at low polynomial degree, cannot deal with complex datasets
 - And at high polynomial degree, creates huge (combinatorial explosion) number of features (slow).
- When using SVM, can use the **kernel trick**
 - Can give same result as if added many polynomial features, without the computational costs of actually adding them.
- The kernel trick is built-in to the `scikit-learn` SVC class.

SVC Polynomial Kernel



```
# using a degree 3 polynomial kernel for a SVC,  
# this implementation will be much faster for large  
# polynomial degrees / large datasets  
from sklearn.svm import SVC
```

```
poly_kernel_svm_clf = Pipeline([  
    ("scaler", StandardScaler()),  
    ("clf", SVC(kernel="poly", degree=3, coef0=1, C=5))  
])
```

```
poly_kernel_svm_clf.fit(X, y)
```

Figure 5: SVC Using degree 3 polynomial kernel (implemented using the kernel trick).

SVM Kernel Regularization

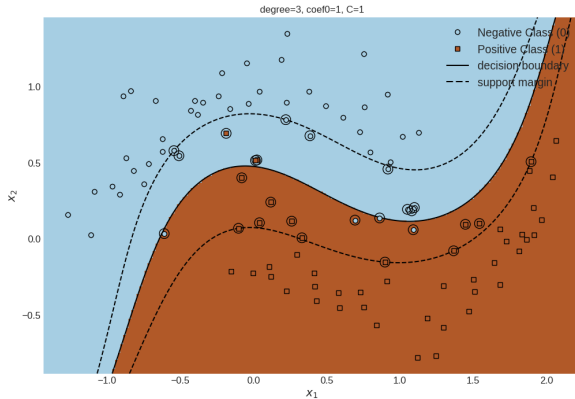


Figure 6: SVC Using degree 3 polynomial kernel, large regularization, if need to underfit.

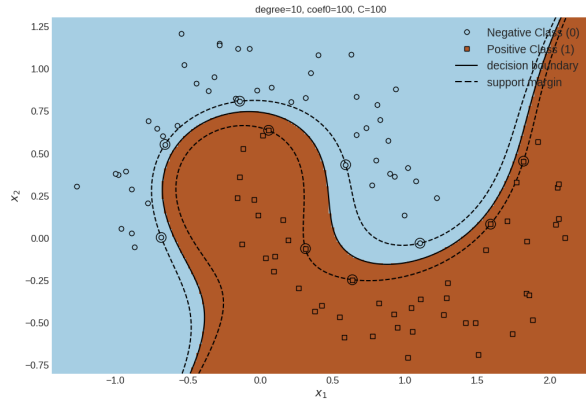
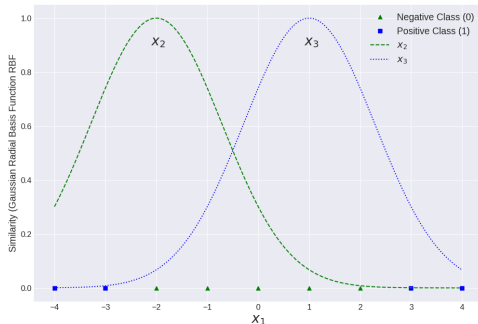


Figure 7: SVC Using degree 10 polynomial kernel, small regularization, if need to overfit.

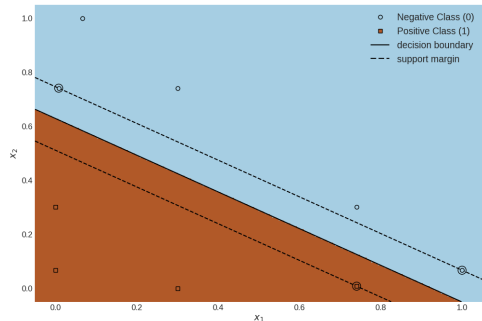
Similarity Features

- Instead of combinations of polynomial terms of features, can compute new nonlinear features using a **similarity function**.
- Measures how much each instance resembles a particular **landmark**

Two landmarks at $x_1 = -2$ and $x_1 = 1$ and Gaussian RBF similarity function.



New computed similarity features x_2 and x_3 are now linearly separable.



Gaussian Radial Basis Function (RBF) Kernel

- Similarity feature method can be used with any ML algorithm, but can be computationally expensive to compute all the additional features.
- SVM **kernel trick** mathematical magic that allows for computation of gaussian kernels for SVM fitting, without actual computation of the new similarity features.

Gaussian Radial Basis Function (RBF) Kernel

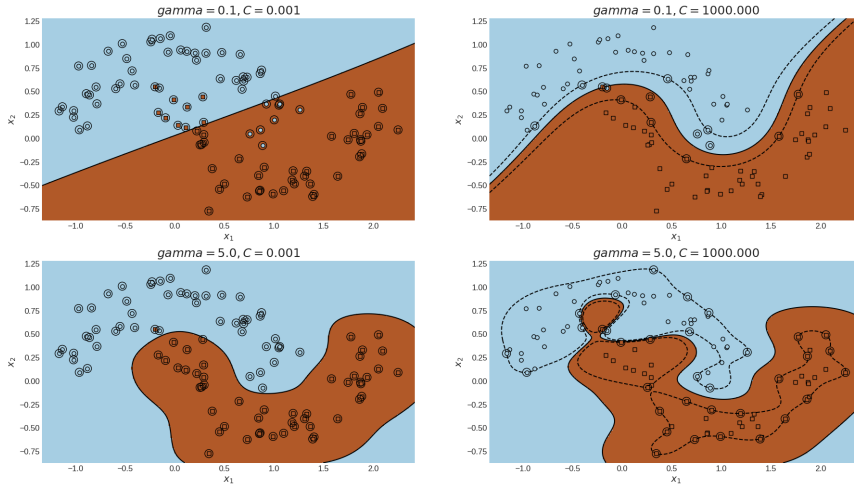


Figure 8: SVM classifiers using Gaussian RBF kernels. (Géron, 2023, pg.299)

SVM Computational Complexity

- `scikit-learn` `LinearSVC` does not support kernel trick, but scales linearly with number of training instances (fast)
 - training time complexity $\mathcal{O}(m \times n)$
- `SVC` class supports **kernel trick**
 - training complexity between $\mathcal{O}(m^2 \times n)$ and $\mathcal{O}(m^3 \times n)$
 - here m is number of training instances, so can be very slow when number of training instances is large
 - scales well with number of features n
- `SGDClassifier` also performs large margin classification by default, using stochastic gradient descent for incremental learning.
 - still a linear classifier
 - so has complexity $\mathcal{O}(m \times n)$, but can do out-of-core learning



SVM Computational Complexity

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$\mathcal{O}(m \times n)$	No	Yes	No
SVC	$\mathcal{O}(m^2 \times n)$ to $\mathcal{O}(m^3 \times n)$	No	Yes	Yes
SGDClassifier	$\mathcal{O}(m \times n)$	Yes	Yes	No

SVM Regression

- Can use SVM for regression, tweak objective function
 - SVM regression fits as many instances as possible **on** the street
 - Instead of fitting largest possible street between classes (while limiting margin violations)
- SVM regression uses hyperparameter `epsilon` or ϵ to control width of street.

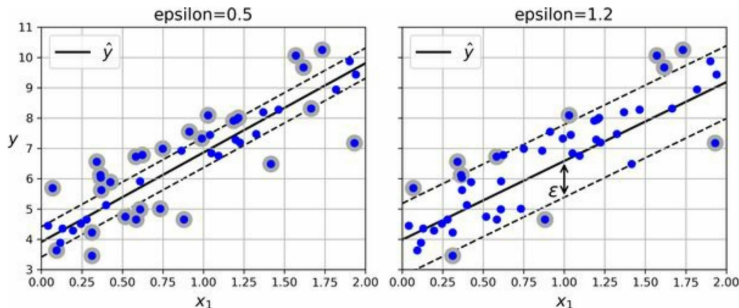


Figure 9: SVM regression, effects of epsilon metaparameter. (Géron, 2023, pg.302)

Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras and tensorflow* (third).
O'Reilly Media, Inc.